Sparc4D: A Compact Explicit 4D Representation for Dynamic Scenes

Di Yang[1] Zhihao Li[2] Yanhai Xiong[1,*] Yufei Wang[2,*,†]
[1]College of William & Mary [2]SparcAI Inc.
dyang06@wm.edu zhihao.li@sparclab.ai yxiong05@wm.edu yufei.wang@sparclab.ai

Abstract

We present Sparc4D, a feed-forward autoencoder that encodes a monocular video with known cameras into a compact explicit 4D scene state. Static features are shared across the clip, while spatially anchored temporal slots compress the time-varying features. Fine-grid appearance codes and sparse source texture preserve spatial detail alongside this temporal bottleneck. Temporal slots use a median $5.0\times$ fewer values than per-frame storage of time-varying features, with negligible change in novel-view quality. A sparse decoder reconstructs 2D Gaussian surfels that provide geometry for rendering and source-texture reprojection. The complete state includes learned features, sparse occupancy, one full source frame and dynamic-region pixels sampled every fourth frame. On random MultiCamVideo windows, it averages 0.67M 32-bit-equivalent values per 32-frame clip. At this size, it retains novel-view quality comparable to MoVieS and higher PSNR than MCV-fine-tuned C4G. Training on synthetic scenes also supports zero-shot reconstruction on DyCheck and Neu3D. After domain adaptation, the frozen encoder supports a ManiSkill3 StackCube policy with 81.3% success from 1,000 demonstrations.

1. Introduction

Feed-forward dynamic-scene reconstruction turns monocular video into a representation that can be rendered from new viewpoints. Storing and reusing that representation requires a balance between state size and reconstruction quality. Pixel-aligned Gaussian methods such as MoVieS [25] can produce large outputs when motion is materialized across a clip. Compact Gaussian predictors such as C4G [16] reduce the number of primitives and trade reconstruction fidelity for size. We study how to encode a posed monocular video into a compact explicit 4D scene state while retaining its novel-view appearance.

Video tokenizers [30, 54] compress image sequences, but their pixel-space outputs do not by themselves provide an explicit 3D rendering interface. Sparse 3D autoencoders [47, 48] provide such structure for individual assets. Dynamic scenes require both temporal variation and spatial detail. Per-frame storage repeats static content and grows with the number of observations, while a low-resolution latent limits the appearance detail that each cell can retain. These information types motivate complementary storage paths: shared static features, compressed time-varying features and fine source appearance.

Sparc4D assigns static and dynamic observations different temporal representations. Estimated depth lifts a posed video into sparse 3D observations, which a shared encoder maps to spatial features. Static and coarse features are stored once per clip. Dynamic features are encoded per frame, then pooled into a small number of temporal slots at each occupied spatial cell. Reading these slots at a cell and timestamp recovers the dynamic features needed by the sparse decoder. Sparse occupancy is stored alongside the features to define the decoded surfaces.

Fine appearance is retained through learned codes and stored source pixels. A fine-grid appearance code supplements the 32^3 latent, while one full source frame and dynamic-region pixels sampled every fourth frame preserve source detail. The decoder reconstructs 2D Gaussian surfels [13], whose geometry supports both explicit rendering and reprojection of the stored pixels. This reprojection follows image-based rendering and depth-image-based video coding [3, 8, 10, 41]. The image loss is computed after texture reprojection, matching training to the complete decoding path.

We evaluate reconstruction quality and state size on MultiCamVideo [2], including a C4G model fine-tuned on the same training split. We also test reduced-output MoVieS variants and zero-shot transfer to DyCheck [11] and Neu3D [20]. Temporal-compression and stored-texture ablations explain how the state balances feature storage and rendering detail. We also analyze reconstruction by region and time. Finally, a manipulation policy trained on frozen

*Corresponding author. †Project lead.
Preprint. Work in progress. 2026 Sep 26th.

encoder features tests their utility for control.

The contributions are:

- A sparse 4D scene representation that shares static features and compresses dynamic features into spatially anchored temporal slots.
- A decoding design that combines fine-grid appearance codes and sparse stored texture through explicit surfel geometry, retaining source detail in novel views.
- A size–quality study showing that temporal slots compress time-varying features with negligible change in novel-view quality, together with comparisons to MoVieS, 4DGT and C4G and tests of real-video transfer and feature utility for control.

2. Related Work

Dynamic scene reconstruction. Dynamic radiance fields model time-varying scenes through deformation [32, 33, 35], scene flow [21], or image-based rendering [22]. Gaussian splatting [15] supports deformable and time-varying primitives [29, 46, 52]. Shape of Motion and MoSca regularize motion using low-dimensional bases or scaffolds [19, 44]. These methods optimize a scene-specific representation. We instead learn an encoder and decoder shared across scenes.

Feed-forward reconstruction. Image-based reconstruction networks predict 3D representations in one pass [4, 6, 12, 49, 56]. VGGT and MonST3R estimate geometry and cameras from static or dynamic inputs [42, 55]. For dynamic reconstruction, L4GM and STORM predict time-dependent Gaussians [37, 51]. MoVieS attaches motion to pixel-aligned Gaussians [25], 4DGT predicts space-time Gaussians from monocular video [50], and C4G uses timestamp-conditioned queries to decode a compact Gaussian set [16]. We compare with the latter three using their public weights and additionally fine-tune C4G on our training split at its original output budget. We compare methods by the size of their encoded scene state and its reconstruction quality.

Scene and video compression. Video tokenizers such as MAGVIT-v2 and Cosmos encode image sequences into spatiotemporal latents [30, 54]. TRELLIS, TRELLIS.2, Sparc3D and XCube use structured or hierarchical sparse representations for 3D assets [23, 38, 47, 48]. SceneTok encodes static scenes into tokens decoded by a diffusion model [1]. We adapt the TRELLIS.2 shape autoencoder to dynamic scenes and add a temporal bottleneck based on learned queries [14, 27]. Structured 4D latents have also been used for robot planning [24]; Section 4.6 relates a policy built on Sparc4D features to their StackCube result.

Image-based rendering. View-dependent texture mapping, unstructured lumigraphs and learned image-based rendering use reference images to texture scene geometry [3, 8, 39, 43]. Depth-image-based video coding similarly combines stored images and geometry to synthesize new views [10, 41]. Our stored texture follows this principle, but uses geometry reconstructed from the stored 4D state rather than transmitted depth maps. For evaluation, we also adopt the distinction between observed and unobserved regions motivated by DyCheck [11].

3. Method

Sparc4D stores a clip as shared static features, spatially anchored dynamic slots and sparse source texture (Fig. 1). The decoder reads this state at an observed time and renders it through explicit surfel geometry. Appendix A.1 summarizes the state and the encoding/decoding procedure.

3.1. Scene observations

The input is a monocular clip of $T = 32$ RGB frames $\{I_t\}$ at 256×256, recorded by one moving *source camera* with known intrinsics and world-to-camera poses. We estimate depth using Depth Anything 3 [26], conditioned on these cameras. A pixel $\mathbf{x}$ is lifted into world coordinates as

$$\mathbf{p}_t(\mathbf{x}) = \mathbf{R}_t^\top \left(D_t(\mathbf{x}) \mathbf{K}^{-1} \tilde{\mathbf{x}} - \mathbf{t}_t \right), \tag{1}$$

where D_t is estimated depth, $(\mathbf{R}_t, \mathbf{t}_t)$ is the camera pose, and $\tilde{\mathbf{x}}$ is the homogeneous pixel coordinate. Each point carries RGB, a depth-derived normal and its position.

Static observations accumulate spatial evidence across the clip, while dynamic observations retain their timestamps. We therefore pool static points and retain dynamic points separately for each frame. To separate them, we test whether another frame sees through a point's expected position. Foreground object masks [58] extend these motion seeds to whole objects, including intervals when an object briefly stops. Appendix A gives the test and mask parameters.

We voxelize the observations inside a scene-adaptive cube at 512^3 resolution. A larger, coarser cube stores static points outside this region. We call the three resulting voxel sets the static, coarse and dynamic *layers* of the scene. An equirectangular environment map stores more distant observations by ray direction [7]. Pixels covered by neither surfels nor the map receive the source video's mean colour. Each occupied voxel stores mean RGB, mean normal and a within-voxel point offset. Octant colour residuals retain sub-voxel variation before encoding.

3.2. Shared static features and temporal slots

We initialize the sparse encoder–decoder from the TRELLIS.2 shape autoencoder [48]. It maps a 512^3 sparse voxel

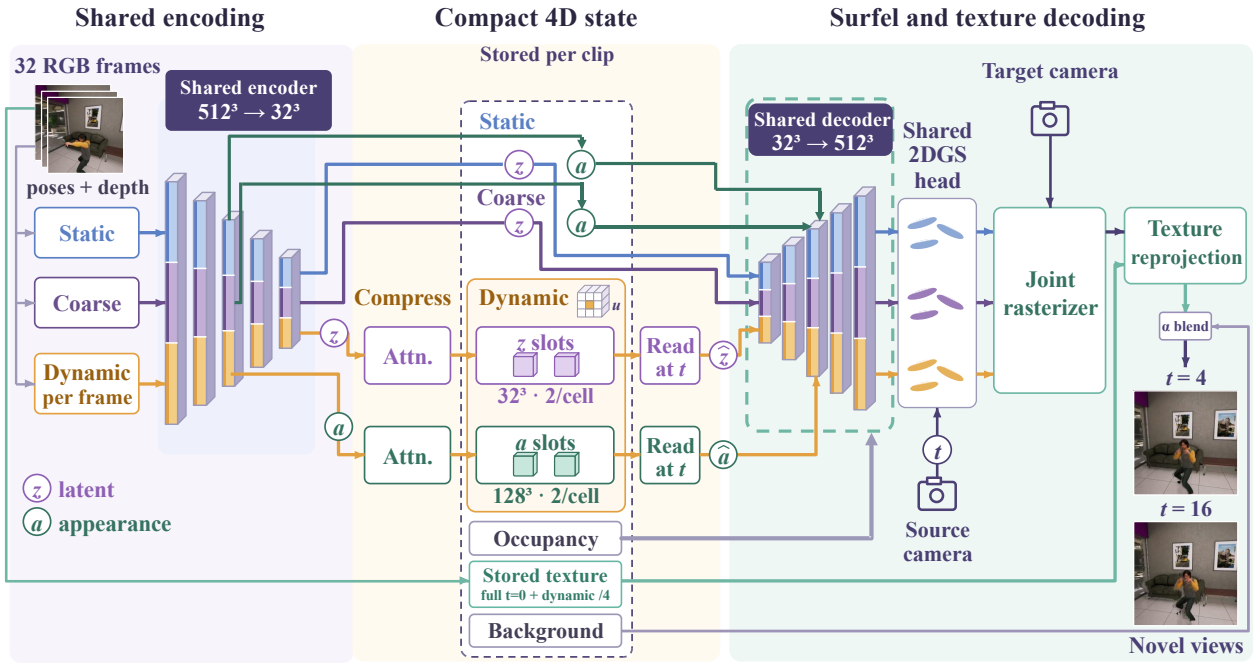


Figure 1. **Overview of Sparc4D.** A shared sparse encoder processes static, coarse and per-frame dynamic observations from a posed monocular clip. Static/coarse codes are stored once. Independent attention modules compress dynamic latent and appearance codes on the 32^3 and 128^3 grids into two temporal slots per cell. The small grid highlights one spatial cell u. At observed time t, cell-local readouts recover the dynamic codes, with appearance injected at 128^3. The three layers merge at joint 2DGS rasterization. Stored texture contains the full first frame and dynamic pixels sampled every fourth frame; depth-consistent reprojection updates colour before background compositing. Decoding uses the stored state and cameras. Photos show MultiCamVideo inputs and complete model outputs; surfels are schematic.

grid to a 32-channel latent on a 32^3 grid. The static, coarse and per-frame dynamic layers are encoded as separate batches with the same network weights. The static and coarse latents are stored once per clip. The decoder uses the input occupancy hierarchy for sparse upsampling; this hierarchy is part of the stored state.

Temporal slots compress the dynamic features at each fixed spatial location. A *cell* is one position of the 32^3 latent grid, as illustrated by the highlighted cell in Fig. 1. Let $\mathcal{U}$ be the union of dynamic cells over the clip, and $\mathbf{z}_{u,t}$ the latent of cell u at time t. We compress the available per-frame features into two slots per cell. We obtain them by multi-head attention pooling with two learnable seed vectors [18], followed by cross-cell mixing and projection:

$$\begin{aligned} \mathbf{k}_{u,t} &= \mathbf{W}\mathbf{z}_{u,t} + \phi_t(t) + \phi_p(u), \\ \mathbf{A}_u &= \mathrm{Attn}(\mathbf{Q} + \phi_p(u), \{\mathbf{k}_{u,t}\}_{t\in\mathcal{T}_u}), \\ \{\mathbf{S}_u\}_{u\in\mathcal{U}} &= \mathrm{Proj}(\mathrm{Mix}(\{\mathbf{A}_u\}_{u\in\mathcal{U}})), \\ \hat{\mathbf{z}}_{u,t} &= \mathrm{Dec}(\phi_t(t) + \phi_p(u), \mathbf{S}_u). \end{aligned} \tag{2}$$

Here $\mathcal{T}_u$ contains the frames in which cell u is occupied, and ϕ_t, ϕ_p embed time and position. The notation $\mathrm{Attn}(\mathbf{q}, X)$ uses $\mathbf{q}$ as queries and learned projections of X as keys and values. Two learned queries $\mathbf{Q}$ first pool features within each cell. Self-attention then mixes the slots across cells, and a projection reduces each slot to 32 channels. To recover a latent, a query at (u, t) attends to the two stored slots of u. The decoded latent $\hat{\mathbf{z}}_{u,t}$ replaces $\mathbf{z}_{u,t}$ in the sparse decoder. The stored dynamic latent contains $2|\mathcal{U}|$ vectors, each with 32 channels.

3.3. Appearance and surfel decoding

A fine-grid appearance code retains spatial detail beyond the 32^3 latent. Each latent cell spans 16^3 input voxels; we supplement it with a 4-channel code on the 128^3 grid. For encoder and decoder features $\mathbf{e}_u$ and $\mathbf{g}_u$ at this resolution,

$$\mathbf{a}_u = \mathbf{W}_e\,\mathrm{LN}(\mathbf{e}_u), \qquad \mathbf{g}_u \leftarrow \mathbf{g}_u + \gamma\mathbf{W}_d\mathbf{a}_u, \tag{3}$$

where LN is layer normalization and γ is a fixed gain. Static/coarse appearance codes are shared across the clip. An independent temporal module applies Eq. 2 to dynamic appearance, storing two 4-channel slots per cell in the 128^3 temporal union. Its readout at an occupied cell and time is $\hat{\mathbf{a}}_{u,t}$.

Each decoded voxel emits four 2D Gaussian surfels (2DGS) [13, 34] with degree-1 spherical-harmonics colour.

The 2DGS head turns the decoder output into centre offsets, scales, rotations and opacity, as residuals from a camera-facing base configuration. The base uses the source camera at time t. The surfels of the three layers are rasterized together with gsplat [53]. Appendix A.3 gives the parameterization. Dynamic decoding uses the occupancy of frame t, so t is one of the 32 input timestamps (an *observed time*).

3.4. Stored source-pixel texture

Stored texture preserves source detail alongside the learned reconstruction of surfaces and appearance. We retain the first source frame in full and refresh dynamic-region pixels every fourth frame. The dynamic region is where the decoded dynamic layer's source-view opacity exceeds 0.5, dilated by two pixels. Pixels already stored in the first frame are not counted again.

For a target camera c at time t, rasterization gives a colour image and median depth $\hat{d}_c$. We back-project a target pixel using $\hat{d}_c$ and project the resulting point into a stored frame. If its projected depth agrees with the depth rendered in that stored frame's source view, we use the sampled stored colour. Otherwise, or if the point falls outside the stored region, we retain the surfel colour. Both depth maps are rendered from the decoded state.

Static pixels use the first source frame. Dynamic pixels use the two stored dynamic frames that bracket t. Each frame passes the same projection and depth test, and the accepted colours are blended by temporal distance. Stored texture changes only colour; geometry and coverage come from the surfels. The projection and consistency test are specified in Appendix A.5.

3.5. Training

For each training sample, we select a 32-frame window and one source camera. Its frames form the input. We supervise frame 0, a random time t and the two stored dynamic frames bracketing t. At each of these times we render the source view and four of the remaining nine cameras. The image loss combines L_1, SSIM and LPIPS and is computed after stored texture is applied; replaced pixels supervise only opacity. Depth and normal losses supervise geometry, while a temporal loss matches inter-frame image differences. The temporal slots also receive feature-reconstruction losses. Occupancy and KL losses follow TRELLIS.2 [17, 48].

Sparc4D is initialized from the pretrained TRELLIS.2 model and trained for 30,000 steps on randomly placed 32-frame windows, with one clip per GPU on eight A800 GPUs. Appendix B gives the full objective, optimization settings, and ablation protocol.

3.6. State size

We count the stored features, sparse occupancy, environment map, background colour and stored texture. The unit is a *32-bit-equivalent value*: feature values count as one each, while occupancy entropy in bits is divided by 32. Each stored 8-bit RGB channel is counted as one 32-bit value. Shared network weights and externally supplied cameras are excluded.

Let $\mathcal{C}$ and $\mathcal{U}$ denote static/coarse cells and dynamic union cells on the 32^3 grid. Their 128^3 counterparts store appearance. With occupancy cost B_{occ} bits, environment texel set $\mathcal{E}$ and additional dynamic-pixel set $\mathcal{P}_{\mathrm{dyn}}$, the reported size is

$$N = 32|\mathcal{C}| + 64|\mathcal{U}| + 4|\mathcal{C}^{128}| + 8|\mathcal{U}^{128}| + B_{\mathrm{occ}}/32 + 4|\mathcal{E}| + 3 + 3 \cdot 256^2 + 3|\mathcal{P}_{\mathrm{dyn}}|. \quad (4)$$

Occupancy is estimated from octree symbols. For dynamic occupancy, we use the smaller estimate from independent frames or consecutive-frame XOR masks. Appendix C details this accounting and gives both stored-state and time-materialized sizes for the compared methods. Table 11 separates the learned state from its stored texture.

4. Experiments

We first evaluate the state size–quality tradeoff on novel-view reconstruction. Ablations then isolate temporal compression, stored texture and feature capacity. Finally, real-video transfer and robotic control test the learned representation beyond synthetic reconstruction.

4.1. Data and protocol

Dataset and split. MultiCamVideo (MCV) [2] contains 3,400 Unreal Engine scenes. Each scene is rendered at four focal lengths by ten synchronized cameras for 81 frames at 15 fps. A scene–focal-length pair forms one clip, giving 13,600 clips. Frames are resized to 256^2.

The held-out test set contains 101 scenes, selected by clustering DINOv2 scene features [31], with all four focal lengths of each scene, giving 404 clips. All MultiCamVideo results in this paper use this test set. Appendix E.1 gives the construction details.

Windows and cameras. The cameras coincide at the start of each video and then move apart. We therefore evaluate two window placements. The *first-32* protocol uses frames 0–31. The *random-window* protocol fixes one randomly sampled start in $[0, 49]$ for each clip. The source camera is fixed by the clip's test-list index modulo ten; the other nine cameras are targets. All methods render every target at all 32 timestamps, giving 116,352 target images per protocol.

Metrics. We report image-averaged PSNR, SSIM [45] and AlexNet-LPIPS [57]. Alongside full-image scores, we evaluate pixels classified as observed by a method-independent visibility mask. The mask uses estimated depth

and foreground segmentation; dynamic foreground requires same-time source visibility. A third setting, *uniform fill*, assigns the same source-mean colour to all methods outside the mask. These settings separate reconstruction on observed regions from differences in unobserved-region rendering. Appendix E.2 specifies the masks and masked metrics.

Baselines and comparison scope. We evaluate MoVieS [25], 4DGT [50] and C4G [16] with released weights. Their training data differ from ours, so these models serve as zero-shot references. For a same-data comparison at a similar representation size, we fine-tune C4G on the MultiCamVideo training split for 30,000 steps, keeping its 2,048-Gaussian output budget. Sparc4D is trained for the same number of steps on the same split. All methods render at 256^2; input settings and coordinate alignment are given in Appendix E.3. Following VideoTok4D [5], #Floats counts the FP32 per-sequence state for 32 frames. Shared network weights are excluded, and time-conditioned Gaussians are materialized at every evaluated timestamp.

4.2. Novel-view reconstruction

On random windows, Sparc4D reaches 17.39 dB with a 0.67M-value state (Table 1). MoVieS obtains 17.26 dB; the paired per-clip difference is $+0.12 \pm 0.18$ dB (95% CI). On first-32, Sparc4D reaches 21.97 dB versus 20.40 dB for MoVieS (Table 2).

On co-visible pixels, Sparc4D obtains 19.02 dB and 0.290 LPIPS; MoVieS obtains 18.55 dB and 0.296. The paired LPIPS difference is -0.005 ± 0.008, so these perceptual scores are comparable. Uniform-fill evaluation preserves the PSNR ordering. With a smaller state than MCV-fine-tuned C4G (0.67M versus 0.92M), Sparc4D has higher full-image PSNR and lower LPIPS. Section 4.4 separates the contributions of learned features and stored texture.

Figures 2 and 3 compare the complete method with the released baselines on fixed examples selected near the median per-clip PSNR difference. The same scenes and crop locations are used for all methods; Appendix E.6 gives the selection procedure.

4.3. Size–quality comparison

We evaluate whether reducing the output of a pixel-aligned reconstructor can reach a similar size–quality tradeoff. We give MoVieS fewer input frames or a lower input resolution, or prune its Gaussians by opacity, and evaluate all variants on the first-32 protocol (Table 2).

With four input frames, MoVieS obtains 20.04 dB with 28.5M output values; its single-frame variant obtains 18.97 dB with 7.1M. Sparc4D reaches 21.97 dB and 0.199 LPIPS with 0.66M values, about $11\times$ smaller than the single-frame variant. Lowering the input resolution or pruning by opacity reduces quality more strongly at comparable sizes.

4.4. Representation analysis

Temporal compression. Table 3 compares temporal slots with per-frame dynamic features. The mean time-varying feature count drops from 354.5K to 79.4K values, reducing the complete state from 0.96M to 0.67M. The per-clip compression factor for these features has a median of $5.0\times$, while paired PSNR changes by only $+0.006 \pm 0.005$ dB. Appendix 12 gives the component-wise storage breakdown and compression factors.

Stored-texture ablation. Table 4 varies only the stored texture of the trained model. Adding stored texture raises PSNR from 16.82 to 17.39 dB and lowers LPIPS from 0.469 to 0.370; most of this improvement comes from the first frame. Shorter intervals change PSNR by at most 0.03 dB but enlarge the state by up to $2.5\times$. Blending the two bracketing frames outperforms both single-frame reuse and no reuse across time.

Feature capacity and design. The slot-capacity ablation holds the checkpoint, training schedule and absence of appearance codes fixed (Table 9). Increasing K from 2 to 10 raises PSNR by 0.04 dB on average (paired 95% CI: ± 0.01 dB), while dynamic-slot storage increases from 21.1K to 105.6K values per clip. The small quality gain in this 12k-step setting motivates the two-slot configuration used by the model.

Each configuration in Table 10 is trained for 10k steps from a shared 20k-step geometry-only checkpoint; Appendix B.1 gives the details.

The appearance pathway improves PSNR by 0.18 dB over geometry only, while increasing the appearance-code capacity adds another 0.06 dB. The motion mask changes PSNR by -0.04 dB, and the combined configuration differs by only $+0.01$ dB from the appearance-code configuration.

Voxelization and appearance. In the region analysis, the dynamic region of a target image is where the current-frame dynamic input voxels project, and the static region is the rest of the scene cube (Appendix E.2). Table 18 locates where dynamic detail is lost. Voxelizing current-frame dynamic pixels at 512^3 lowers source-view dynamic PSNR from 27.27 dB (direct pixel rendering) to 18.12 dB. The learned state recovers 20.75 dB without stored texture. At target views, MoVieS leads in dynamic regions (16.76 versus 16.36 dB) by transporting primitives from multiple input frames to time t (Sec. 5).

Method	Trained on MCV	#Floats	Full image PSNR↑	Full image SSIM↑	Full image LPIPS↓	AbsRel↓	Co-visible PSNR↑	Co-visible SSIM↑	Co-visible LPIPS↓	Uniform fill PSNR↑	Uniform fill SSIM↑	Uniform fill LPIPS↓
MoVieS	–	719M	*17.26*	*0.512*	**0.366**	0.615 (0.272)	*18.55*	*0.530*	*0.296*	*16.43*	*0.498*	*0.411*
4DGT	–	26.9M	14.35	0.403	0.550	0.640 (0.370)	15.14	0.411	0.538	14.49	0.412	0.546
C4G	–	*0.92M*	13.64	0.369	0.596	0.543 (0.450)	14.76	0.394	0.566	14.21	0.393	0.572
C4G	✓	*0.92M*	16.51	0.466	0.584	*0.356 (0.346)*	17.14	0.470	0.561	15.65	0.444	0.564
Sparc4D	✓	**0.67M**	**17.39**	**0.519**	*0.370*	**0.259 (0.170)**	**19.02**	**0.541**	**0.290**	**16.62**	**0.509**	**0.405**

Table 1. **Novel-view synthesis on MultiCamVideo** (404 random-window test clips, 9 target cameras × 32 frames, 256^2). Best and second-best results are shown in bold and underlined italics, respectively. Sparc4D denotes the complete method, including stored texture. #Floats follows [5]: time-conditioned Gaussian outputs are materialized at every timestamp, and other methods count their per-sequence state (Eq. 4 for ours); shared network weights are excluded. C4G is shown with released weights and after fine-tuning for 30,000 steps on the MCV training split, the number of steps used to train Sparc4D. AbsRel uses DA3 pseudo ground truth, also our input depth; parentheses give median-scale-aligned error. Co-visible metrics use the shared visibility mask. Uniform fill assigns the same colour outside that mask for all methods.

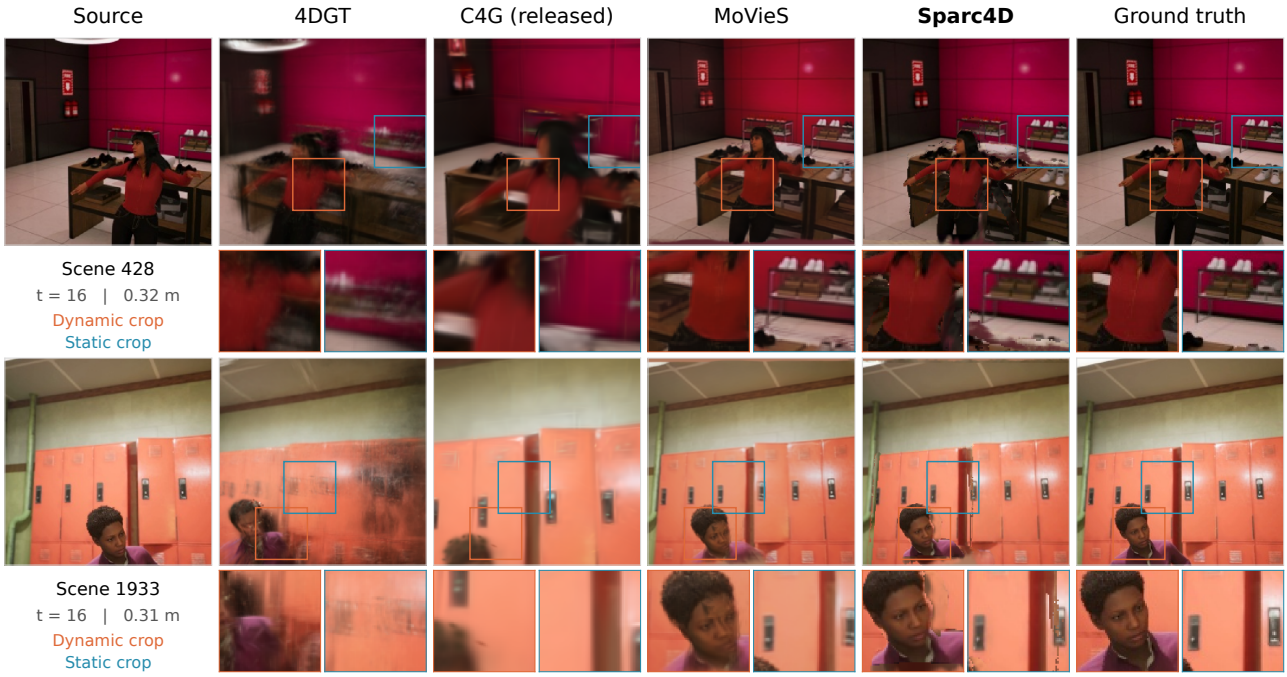


Figure 2. **Qualitative comparison, first-32 protocol.** Two fixed test scenes at frame 16, using the median-distance target camera. Orange and blue boxes mark dynamic and static 64^2 crops, enlarged below at identical coordinates for all methods. Sparc4D is the complete model; baseline images use released weights. Camera distance is shown in metres.

Static and dynamic reconstruction. On first-32, stored texture matters most in static regions: it raises PSNR from 21.43 to 25.41 dB, above the 21.79 dB of MoVieS (Table 17). In dynamic regions, refreshing the stored pixels every fourth frame raises PSNR from 18.36 to 19.32 dB. It exceeds MoVieS by 0.34 ± 0.32 dB in paired PSNR, with similar LPIPS (0.153 versus 0.150).

Texture benefit over time. The benefit of stored texture decreases later in the window (Table 19). The same trend holds when all source frames are stored. Growing source–target separation and reprojection error therefore appear to limit the benefit more than the spacing of stored frames. Dynamic refresh helps primarily at earlier timestamps.

Appendix G shows that Sparc4D preserves the DA3 depth structure more closely than the evaluated baselines.

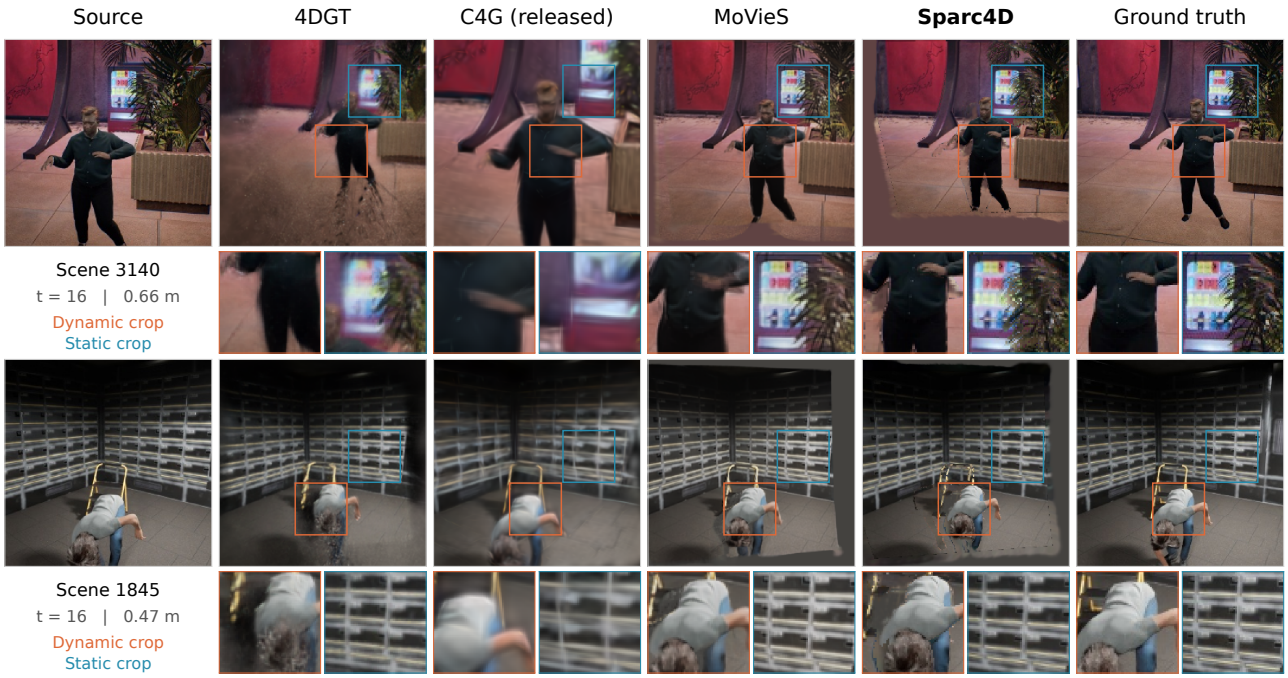


Figure 3. **Qualitative comparison, random-window protocol.** Same layout and method settings as Fig. 2.

Table 2. **Size and quality** on the first-32 protocol. MoVieS uses its native motion-parameter output count here, rather than the time-materialized count in Table 1; Appendix C explains both. Our size includes all stored texture. MoVieS variants use f inputs at resolution r, or retain the k highest-opacity Gaussians.

Method	Size	PSNR↑	SSIM↑	LPIPS↓
MoVieS, 32f, 224	228M	20.40	0.643	0.207
MoVieS, 8f, 224	57.0M	20.28	0.642	0.203
MoVieS, 4f, 224	28.5M	20.04	0.628	*0.201*
MoVieS, 2f, 224	14.3M	19.61	0.619	0.219
MoVieS, 1f, 224	7.1M	18.97	0.619	0.232
MoVieS, 8f, 112	14.3M	16.90	0.435	0.558
MoVieS, top 200k	28.4M	16.25	0.485	0.491
MoVieS, top 50k	7.1M	14.96	0.434	0.642
Sparc4D	**0.66M**	**21.97**	**0.686**	**0.199**

Table 3. **Temporal compression ablation** on random windows. Both variants use the same trained weights and stored texture, which is included in the total state. "Per-frame" bypasses temporal-slot compression for dynamic latent and appearance features; dynamic occupancy is unchanged. Sizes are mean 32-bit-equivalent values per 32-frame clip.

	Storage ↓		Reconstruction		
Representation	Time-varying features	Total state	PSNR↑	Dyn. PSNR↑	Dyn. LPIPS↓
Per-frame	354.5K	0.96M	17.38	16.23	0.263
Temporal slots	**79.4K**	**0.67M**	17.39	16.24	0.262

Table 4. **Stored-texture ablation** (404 random-window clips). All rows use the same 30k-step backbone. Interval is the spacing between stored dynamic-texture frames; the first full source frame is retained except in "w/o stored texture."

Configuration	Size	PSNR↑	LPIPS↓	Dyn. LPIPS↓
w/o stored texture	0.17M	16.82	0.469	0.345
w/o dynamic texture	0.36M	17.31	0.384	0.292
Dynamic texture interval: 1	1.69M	17.42	0.367	0.258
Dynamic texture interval: 2	1.01M	17.41	0.368	0.260
Dynamic texture interval: 8	0.50M	17.34	0.371	0.266
Interval 4, no reuse across time	0.67M	17.33	0.380	0.284
Interval 4, preceding frame only	0.67M	17.35	0.370	0.263
Sparc4D (interval: 4)	0.67M	17.39	0.370	0.262

4.5. Transfer to real videos

We evaluate zero-shot transfer to DyCheck iPhone [11] and Neu3D [20] (Table 5) without real-data fine-tuning. DyCheck uses official co-visibility masks, whereas Neu3D uses full-image metrics; Appendix E.4 and Appendix E.5 give the full protocols.

On DyCheck, Sparc4D reaches 13.82 dB mPSNR, close to MoVieS at 13.95 dB, with higher mLPIPS (0.543 versus 0.488). On Neu3D, it reaches 16.57 dB across nine target cameras and 18.54 dB on cam00; 4DGT reaches 17.33 and 19.51 dB.

Table 6 isolates the effect of stored texture using identical model weights and inputs. On both real datasets, addi-

Method	Training data	#Floats	DyCheck mPSNR↑	DyCheck mSSIM↑	DyCheck mLPIPS↓	Neu3D (9 targets) PSNR↑	Neu3D (9 targets) SSIM↑	Neu3D (9 targets) LPIPS↓	Neu3D cam00 PSNR↑
MoVieS	8 public datasets (∼174K scenes)	719M	**13.95**	*0.345*	**0.488**	15.61	0.445	0.304	17.01
4DGT	∼300 h real monocular video	26.9M	11.84	0.277	0.599	**17.33**	**0.530**	**0.281**	**19.51**
C4G	Spring, Kubric, RE10K	*0.92M*	13.52	**0.351**	0.634	11.78	0.285	0.588	13.41
C4G (MCV fine-tuned)	+ MultiCamVideo	*0.92M*	13.34	0.340	0.747	15.75	0.443	0.579	16.21
Sparc4D	MultiCamVideo (synthetic)	**0.67M**	*13.82*	0.323	*0.543*	*16.57*	*0.465*	*0.291*	*18.54*

Table 5. **Zero-shot generalization to real videos.** Best and second-best results are shown in bold and underlined italics. None of the methods is fine-tuned on DyCheck or Neu3D. DyCheck uses official co-visibility evaluation; Neu3D reports nine target cameras and cam00 separately. #Floats repeats the MCV reference size for context rather than a measured real-data average.

Table 6. **Stored texture on real videos.** All rows use the same model weights and inputs. DyCheck uses official masked metrics; Neu3D uses full-image metrics over nine target cameras.

Configuration	DyCheck mPSNR↑	DyCheck mLPIPS↓	Neu3D PSNR↑	Neu3D LPIPS↓
w/o stored texture	14.16	0.617	16.65	0.394
w/o dynamic texture	13.98	0.555	16.59	0.319
Sparc4D	13.82	0.543	16.57	0.291

Table 7. **StackCube success and scene size.** Tokens are active cells per timestamp; d is their feature dimension. Protocols and counting details are in Appendix I.

Method	Demos	Tokens	d	Success
Li et al. [24]	1,000	∼5,700	8	84.0%
Sparc4D features	1,000	661	32	81.3%
+ additional demos	3,000	661	32	84.0%

tional texture progressively improves LPIPS while slightly reducing PSNR. Appendix F shows selected zero-shot reconstructions and aligned detail crops.

4.6. Downstream manipulation

We evaluate the learned scene features on StackCube in ManiSkill3 [40], where a robot must grasp the red cube and stack it on the green cube (Fig. 4). After adapting the autoencoder to ManiSkill scenes, we freeze its encoder and train a flow-matching action policy on static features and the dynamic latents of the three most recent frames. Four calibrated RGB-D views provide the observations; robot masks and causal SAM 2 tracking [36] separate the static and dynamic layers. This control interface uses per-frame latents without temporal-slot compression or stored texture.

With 1,000 demonstrations, the policy reaches 81.3% success, close to the 84.0% reported by Li et al. [24]; adding 2,000 demonstrations raises it to 84.0% (Table 7). Our scene representation averages 426 static and 236 dynamic tokens, with the static features shared across the observation history. The rollout in Fig. 4 illustrates control from these features.

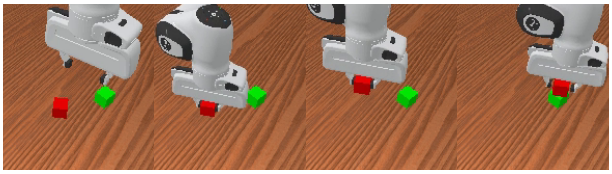


Figure 4. **StackCube manipulation.** A successful rollout of the policy trained on 1,000 demonstrations, shown from one fixed camera. The robot grasps the red cube and places it on the green cube.

5. Limitations

Dynamic decoding uses per-frame occupancy and supports the observed timestamps. Recovering surfaces hidden from the source camera at a query time would require transporting dynamic content across frames. Cross-time texture reprojection relies on locally consistent geometry; depth errors and fast motion can produce incorrect matches. State size depends on occupied volume and dynamic image area, and the entropy-based ledger estimates representation size rather than an implemented codec's bitrate.

6. Conclusion

Sparc4D represents a monocular video through shared static features, spatially anchored temporal slots and fine source appearance. This information split yields a sparse explicit 4D state averaging 0.67M values, with competitive novel-view reconstruction against the evaluated feed-forward methods. Temporal slots reduce time-varying feature storage with negligible change in complete-model rendering quality. Synthetic training transfers to DyCheck and Neu3D, while adapted, frozen encoder features also support StackCube manipulation.

References

[1] Mohammad Asim, Christopher Wewer, and Jan Eric Lenssen. SceneTok: A compressed, diffusable token space for 3D scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[2] Jianhong Bai, Menghan Xia, Xiao Fu, Xintao Wang, Lianrui Mu, Jinwen Cao, Zuozhu Liu, Haoji Hu, Xiang Bai, Pengfei Wan, and Di Zhang. ReCamMaster: Camera-controlled generative rendering from a single video. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.

[3] Chris Buehler, Michael Bosse, Leonard McMillan, Steven Gortler, and Michael Cohen. Unstructured lumigraph rendering. In *Proceedings of SIGGRAPH*, pages 425–432, 2001.

[4] David Charatan, Sizhe Lester Li, Andrea Tagliasacchi, and Vincent Sitzmann. pixelSplat: 3D Gaussian splats from image pairs for scalable generalizable 3D reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[5] Xinyi Chen, Hanxin Zhu, Xijun Wang, Xingrui Wang, Sen Liang, Xin Li, and Zhibo Chen. VideoTok4D: A 4D-aware video tokenizer for compact world representation. *arXiv preprint arXiv:2609.12874*, 2026.

[6] Yuedong Chen, Haofei Xu, Chuanxia Zheng, Bohan Zhuang, Marc Pollefeys, Andreas Geiger, Tat-Jen Cham, and Jianfei Cai. MVSplat: Efficient 3D Gaussian splatting from sparse multi-view images. In *European Conference on Computer Vision (ECCV)*, 2024.

[7] Paul Debevec. Rendering synthetic objects into real scenes: Bridging traditional and image-based graphics with global illumination and high dynamic range photography. In *Proceedings of SIGGRAPH*, pages 189–198, 1998.

[8] Paul E. Debevec, Camillo J. Taylor, and Jitendra Malik. Modeling and rendering architecture from photographs: A hybrid geometry- and image-based approach. In *Proceedings of SIGGRAPH*, pages 11–20, 1996.

[9] David Eigen, Christian Puhrsch, and Rob Fergus. Depth map prediction from a single image using a multi-scale deep network. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[10] Christoph Fehn. Depth-image-based rendering (DIBR), compression, and transmission for a new approach on 3D-TV. In *Stereoscopic Displays and Virtual Reality Systems XI*, pages 93–104, 2004.

[11] Hang Gao, Ruilong Li, Shubham Tulsiani, Bryan Russell, and Angjoo Kanazawa. Monocular dynamic view synthesis: A reality check. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[12] Yicong Hong, Kai Zhang, Jiuxiang Gu, Sai Bi, Yang Zhou, Difan Liu, Feng Liu, Kalyan Sunkavalli, Trung Bui, and Hao Tan. LRM: Large reconstruction model for single image to 3D. In *International Conference on Learning Representations (ICLR)*, 2024.

[13] Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian splatting for geometrically accurate radiance fields. In *SIGGRAPH 2024 Conference Papers*, 2024.

[14] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning (ICML)*, 2021.

[15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42 (4), 2023.

[16] Mungyeom Kim, Minkyeong Jeon, Honggyu An, Jaewoo Jung, Hyuna Ko, Jisang Han, Hyeonseo Yu, Donghwan Shin, Sunghwan Hong, Takuya Narihira, et al. Learning global motion with compact Gaussians for feed-forward 4D reconstruction. *arXiv preprint arXiv:2605.31595*, 2026.

[17] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations (ICLR)*, 2014.

[18] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning (ICML)*, 2019.

[19] Jiahui Lei, Yijia Weng, Adam W. Harley, Leonidas Guibas, and Kostas Daniilidis. MoSca: Dynamic Gaussian fusion from casual videos via 4D motion scaffolds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[20] Tianye Li, Mira Slavcheva, Michael Zollhoefer, Simon Green, Christoph Lassner, Changil Kim, Tanner Schmidt, Steven Lovegrove, Michael Goesele, Richard Newcombe, and Zhaoyang Lv. Neural 3D video synthesis from multi-view video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[21] Zhengqi Li, Simon Niklaus, Noah Snavely, and Oliver Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[22] Zhengqi Li, Qianqian Wang, Forrester Cole, Richard Tucker, and Noah Snavely. DynIBaR: Neural dynamic image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[23] Zhihao Li, Yufei Wang, Heliang Zheng, Yihao Luo, and Bihan Wen. Sparc3d: Sparse representation and construction for high-resolution 3d shapes modeling. *Advances in Neural Information Processing Systems*, 38:118582–118600, 2026.

[24] Zhiyi Li, Peilin Wu, Xiaoshen Han, Ruojin Cai, and Yilun Du. Structured 4d latent predictive model for robot planning. *arXiv preprint arXiv:2607.01166*, 2026.

[25] Chenguo Lin, Yuchen Lin, Panwang Pan, Yifan Yu, Tao Hu, Honglei Yan, Katerina Fragkiadaki, and Yadong Mu. MoVieS: Motion-aware 4D dynamic view synthesis in one second. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.

[26] Haotong Lin, Sili Chen, Jun Hao Liew, Donny Y. Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[27] Francesco Locatello, Dirk Weissenborn, Thomas Unterthiner, Aravindh Mahendran, Georg Heigold, Jakob

Uszkoreit, Alexey Dosovitskiy, and Thomas Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[29] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3D Gaussians: Tracking by persistent dynamic view synthesis. In *International Conference on 3D Vision (3DV)*, 2024.

[30] NVIDIA. Cosmos world foundation model platform for physical AI. *arXiv preprint arXiv:2501.03575*, 2025.

[31] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024.

[32] Keunhong Park, Utkarsh Sinha, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Steven M. Seitz, and Ricardo Martin-Brualla. Nerfies: Deformable neural radiance fields. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.

[33] Keunhong Park, Utkarsh Sinha, Peter Hedman, Jonathan T. Barron, Sofien Bouaziz, Dan B. Goldman, Ricardo Martin-Brualla, and Steven M. Seitz. HyperNeRF: A higher-dimensional representation for topologically varying neural radiance fields. *ACM Transactions on Graphics*, 40(6), 2021.

[34] Hanspeter Pfister, Matthias Zwicker, Jeroen van Baar, and Markus Gross. Surfels: Surface elements as rendering primitives. In *Proceedings of SIGGRAPH*, pages 335–342, 2000.

[35] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural radiance fields for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[36] Nikhila Ravi et al. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024.

[37] Jiawei Ren, Kevin Xie, Ashkan Mirzaei, Hanxue Liang, Xiaohui Zeng, Karsten Kreis, Ziwei Liu, Antonio Torralba, Sanja Fidler, Seung Wook Kim, and Huan Ling. L4GM: Large 4D Gaussian reconstruction model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[38] Xuanchi Ren, Jiahui Huang, Xiaohui Zeng, Ken Museth, Sanja Fidler, and Francis Williams. XCube: Large-scale 3D generative modeling using sparse voxel hierarchies. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[39] Gernot Riegler and Vladlen Koltun. Free view synthesis. In *European Conference on Computer Vision (ECCV)*, 2020.

[40] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. ManiSkill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2025.

[41] Gerhard Tech, Ying Chen, Karsten Müller, Jens-Rainer Ohm, Anthony Vetro, and Ye-Kui Wang. Overview of the multiview and 3D extensions of high efficiency video coding. *IEEE Transactions on Circuits and Systems for Video Technology*, 26(1):35–49, 2016.

[42] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. VGGT: Visual geometry grounded transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[43] Qianqian Wang, Zhicheng Wang, Kyle Genova, Pratul P. Srinivasan, Howard Zhou, Jonathan T. Barron, Ricardo Martin-Brualla, Noah Snavely, and Thomas Funkhouser. IBRNet: Learning multi-view image-based rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[44] Qianqian Wang, Vickie Ye, Hang Gao, Jake Austin, Zhengqi Li, and Angjoo Kanazawa. Shape of motion: 4D reconstruction from a single video. *arXiv preprint arXiv:2407.13764*, 2024.

[45] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[46] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4D Gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[47] Jianfeng Xiang, Zelong Lv, Sicheng Xu, Yu Deng, Ruicheng Wang, Bowen Zhang, Dong Chen, Xin Tong, and Jiaolong Yang. Structured 3D latents for scalable and versatile 3D generation. *arXiv preprint arXiv:2412.01506*, 2024.

[48] Jianfeng Xiang, Xiaoxue Chen, Sicheng Xu, Ruicheng Wang, Zelong Lv, Yu Deng, Hongyuan Zhu, Yue Dong, Hao Zhao, Nicholas Jing Yuan, and Jiaolong Yang. Native and compact structured latents for 3D generation. *Tech report*, 2025.

[49] Haofei Xu, Songyou Peng, Fangjinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and Marc Pollefeys. DepthSplat: Connecting Gaussian splatting and depth. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[50] Zhen Xu, Zhengqin Li, Zhao Dong, Xiaowei Zhou, Richard Newcombe, and Zhaoyang Lv. 4DGT: Learning a 4D Gaussian transformer using real-world monocular videos. *arXiv preprint arXiv:2506.08015*, 2025.

[51] Jiawei Yang, Jiahui Huang, Yuxiao Chen, Yan Wang, Boyi Li, Yurong You, Apoorva Sharma, Maximilian Igl, Peter Karkus, Danfei Xu, Boris Ivanovic, Yue Wang, and Marco Pavone. STORM: Spatio-temporal reconstruction model for large-scale outdoor scenes. In *International Conference on Learning Representations (ICLR)*, 2025.

[52] Ziyi Yang, Xinyu Gao, Wen Zhou, Shaohui Jiao, Yuqing Zhang, and Xiaogang Jin. Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[53] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, Brent Yi, Zhuoyang Pan, Otto Seiskari, Jianbo Ye, Jeffrey Hu, Matthew Tancik, and Angjoo Kanazawa. gsplat: An open-source library for Gaussian splatting. *Journal of Machine Learning Research*, 26(34):1–17, 2025.

[54] Lijun Yu, José Lezama, Nitesh B. Gundavarapu, Luca Versari, Kihyuk Sohn, David Minnen, Yong Cheng, Agrim Gupta, Xiuye Gu, Alexander G. Hauptmann, Boqing Gong, Ming-Hsuan Yang, Irfan Essa, David A. Ross, and Lu Jiang. Language model beats diffusion – tokenizer is key to visual generation. In *International Conference on Learning Representations (ICLR)*, 2024.

[55] Junyi Zhang, Charles Herrmann, Junhwa Hur, Varun Jampani, Trevor Darrell, Forrester Cole, Deqing Sun, and Ming-Hsuan Yang. MonST3R: A simple approach for estimating geometry in the presence of motion. In *International Conference on Learning Representations (ICLR)*, 2025.

[56] Kai Zhang, Sai Bi, Hao Tan, Yuanbo Xiangli, Nanxuan Zhao, Kalyan Sunkavalli, and Zexiang Xu. GS-LRM: Large reconstruction model for 3D Gaussian splatting. In *European Conference on Computer Vision (ECCV)*, 2024.

[57] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[58] Peng Zheng, Dehong Gao, Deng-Ping Fan, Li Liu, Jorma Laaksonen, Wanli Ouyang, and Nicu Sebe. Bilateral reference for high-resolution dichotomous image segmentation. *CAAI Artificial Intelligence Research*, 3, 2024.

A. Observation preprocessing and decoding

A.1. Stored state and encoding/decoding procedure

Table 8 summarizes the information retained after encoding. The latent and appearance grids use independent temporal modules; a cell indexes a fixed spatial position on its respective grid. Their attention modules operate at width 256, then project to the stored channel widths of 32 and 4. Source and target cameras are supplied externally. Cube origins and scales define the grid-to-world transforms; their metadata cost is omitted from the size ledger in Appendix C.

Encode. Given the RGB clip and its source cameras:

1. Estimate depth, lift the observations, separate static and dynamic points, and voxelize the three scene layers.
2. Apply the shared encoder to the static, coarse and per-frame dynamic layers. Collect latent features, appearance codes and the sparse occupancy hierarchies.
3. Retain static/coarse codes once. Independently pool dynamic latent and appearance features into two slots per cell on their temporal unions.
4. Decode the source views at the stored timestamps to obtain dynamic opacity masks. Retain the full first RGB frame and the selected dynamic pixels every fourth frame, using the mask rule in Sec. 3.4.
5. Retain the environment map and background colour alongside the feature state. The cube layout defines its world-coordinate interpretation.

Decode. Given the stored state, cameras and an observed timestamp t:

1. Read dynamic latent and appearance slots at the occupied cells of t.
2. Decode static, coarse and dynamic features with their occupancy hierarchies; inject the appearance codes at 128^3.
3. Predict surfels using the source camera at t for the base configuration. Jointly rasterize the three layers in the target view to obtain colour, depth and opacity.
4. Render source-view depths from the same state for the relevant stored frames. Apply the projection and depth-consistency test in Eq. 7, with temporal blending for dynamic texture.
5. Composite the textured rendering with the environment map and mean-colour background.

A.2. Observation preprocessing

Depth and points. We run pose-conditioned Depth Anything 3 on overlapping 32-frame chunks of the full video. Consecutive chunks are aligned in scale using the median depth ratio on their overlap. Normals are computed from depth gradients, and sky pixels flagged by the depth model are excluded from the point cloud.

Motion seeds. Let $(\mathbf{x}', \zeta')$ be the projection of $\mathbf{p}_t(\mathbf{x})$ into another frame t'. The point is marked dynamic if, for some $|t - t'| \in \{1, 2, 4, 8, 16\}$,

$$\frac{D_{t'}(\mathbf{x}') - \zeta'}{\max(\zeta', 0.05\,\mathrm{m})} > 0.08. \tag{5}$$

We set $D_{t'} = \infty$ on sky. Connected components smaller than 48 pixels are removed, and the mask is dilated by four pixels.

Object masks. RMBG-2.0 (BiRefNet [58]) provides foreground components. An 8-connected component c is selected if it contains at least $\max(16, 0.02|c|)$ motion-seed pixels and covers at most 35% of the image. Selections propagate forward and backward to components overlapping a selected region by at least 30% of their area. Selected regions are dilated by two pixels.

Spatial layout. The scene cube has side length $s = \mathrm{clip}(3\tilde{d}, 4\,\mathrm{m}, 32\,\mathrm{m})$, where $\tilde{d}$ is median camera-to-point distance. Its centre is the median of the nearest 30% of points. A 512^3 grid gives voxel widths of approximately 0.8–6.3 cm. A concentric 64 m cube at the same grid resolution stores outlying static points with 12.5 cm voxels. Points beyond that cube contribute to a 1024×512 equirectangular map by viewing direction relative to the camera-trajectory centre. Only sampled texels are stored.

Voxel features. Each voxel has nine base features: mean RGB, mean normal and mean within-voxel point offset. For each of eight octants we append the difference between its mean RGB and the voxel mean, plus an occupancy flag.

A.3. Surfel parameterization

Each output voxel $\mathbf{v}$ (integer coordinates, voxel size h, cube origin $\mathbf{o}$) emits four 2D Gaussian surfels [13, 34] with degree-1 spherical-harmonics colour. The head predicts residuals $(\boldsymbol{\delta}, \boldsymbol{\ell}, \hat{\mathbf{q}}, \omega)$ around a base configuration that depends on the source camera centre $\mathbf{c}_t^{\mathrm{src}}$ at the rendered time t:

$$\begin{aligned}
\boldsymbol{\mu} &= \mathbf{o} + h\left(\mathbf{v} + \tfrac{1}{2} + \tanh\boldsymbol{\delta}\right), \qquad \alpha = \sigma(\omega + 2), \\
\mathbf{s} &= \mathrm{cap}\left(\rho\,\tfrac{\mathrm{softplus}(\boldsymbol{\ell}+b)}{\mathrm{softplus}(b)}\right), \qquad \rho = \frac{0.55\,\|\mathbf{c}_t^{\mathrm{src}} - \bar{\mathbf{v}}\|}{f}, \\
\mathbf{q} &= \mathrm{norm}\left(\hat{\mathbf{q}} + \mathbf{1}_w\right) \otimes \mathbf{q}_{\mathrm{face}}\left(\mathbf{c}_t^{\mathrm{src}} - \bar{\mathbf{v}}\right),
\end{aligned} \tag{6}$$

where $\bar{\mathbf{v}}$ is the voxel centre and ρ is the world-space footprint of 0.55 pixels at focal length f. We set $b = \log(e - 1)$ and $\mathrm{cap}(s) = s\,s_{\max}/(s + s_{\max})$ with $s_{\max}$ four voxels; $\mathbf{1}_w$ is the identity quaternion, and $\mathbf{q}_{\mathrm{face}}(\mathbf{n})$ turns the surfel normal towards $\mathbf{n}$. With a zero-initialized output layer, each voxel starts as a camera-facing disk of about one pixel.

Table 8. **Stored-state schema.** Features occupy sparse cells on the indicated grids. Static/coarse features are shared across the clip, while dynamic slots are stored on the temporal union of occupied cells. Dynamic occupancy retains the hierarchy at each observed timestamp. The complete state includes stored texture and background information; cameras and shared network weights are external to the state-size accounting.

Component	Grid	Stored form	Decoding role
Static/coarse latent $\mathbf{z}$	32^3	32 channels per occupied cell, once per clip	Shared spatial features
Dynamic latent slots	32^3	Two 32-channel vectors per union cell	Time-varying features read at (u, t)
Static/coarse appearance $\mathbf{a}$	128^3	Four channels per occupied cell, once per clip	Fine-grid appearance injection
Dynamic appearance slots	128^3	Two 4-channel vectors per union cell	Time-varying appearance read at (u, t)
Sparse occupancy	32^3–512^3	Static/coarse hierarchy and per-frame dynamic hierarchies	Sparse upsampling at an observed time
Stored source texture	256^2 images	Full frame 0 and dynamic pixels at $4, 8, \ldots, 28$	Colour sampling through decoded geometry
Environment/background	Ray directions	Environment texels and source-mean RGB	Far and uncovered regions

A.4. Appearance and slot modules

$\mathbf{W}_d$ and the octant-feature input projection are zero-initialized and scaled by a fixed gain $\gamma = 10$, so both residual pathways start at zero. $\mathbf{W}_e$ uses the default initialization. Gradients in the gained layers are scaled by $1/\gamma^2$ before clipping.

The temporal modules use width 256 and eight attention heads. Two learned queries pool the observed tokens of each cell. Three self-attention blocks mix all slots, followed by projection to the stored channel dimension. Three cross-attention blocks decode each (u, t) query. Latent slots have 32 channels; appearance slots have 4. Both receive the normalized feature-matching losses in Appendix B.

A.5. Stored-texture reprojection

Additional-keyframe ablations store k full frames at $\mathcal{K} = \{\mathrm{round}(31i/(k-1))\}_{i=0}^{k-1}$.

Given target depth $\hat{d}_c$, a pixel is back-projected to $\mathbf{X} = \pi_c^{-1}(\mathbf{x}, \hat{d}_c(\mathbf{x}))$ and projected into stored frame τ as $(\mathbf{y}, \zeta) = \pi_\tau(\mathbf{X})$, where π denotes calibrated camera projection and ζ is the depth of $\mathbf{X}$ in frame τ. We accept the sampled colour $I_\tau(\mathbf{y})$ when

$$|\zeta - \hat{d}_\tau(\mathbf{y})| < 0.03\zeta + 0.02\,\mathrm{m}, \tag{7}$$

where $\hat{d}_\tau$ is rendered from the same decoded state in the source view of frame τ. Otherwise the surfel colour is retained. Static pixels try keyframes in order of source-camera distance to the target camera. From full keyframes, pixels where the dynamic layer alone has opacity above 0.5 take colour only at the keyframe's own timestamp. At other times they use the dynamic texture below.

Dynamic stored texture. Dynamic texture is stored at $t = 4, 8, \ldots, 28$. A dynamic pixel at time t blends the stored frames $\tau^- \le t < \tau^+$ by temporal proximity. Only samples that satisfy Eq. 7 and have target opacity above 0.5 are used; source depth is rendered at each stored timestamp. Frame 0 serves as τ^- for $t < 4$, and only τ^- is available for $t > 28$. Texture is applied before environment-map compositing.

B. Training details

The supervised timestamps are $\{0, t, \tau^-, \tau^+\}$. If two coincide or τ^+ falls outside the window, random timestamps fill the gap. The four target cameras are drawn at random from the nine non-source cameras. The loss is

$$\begin{aligned}\mathcal{L} &= \tfrac{1}{4}\textstyle\sum_q \mathcal{L}_q + 0.5\,\mathcal{L}_{\Delta t} + \mathcal{L}_{\text{lat}} + \mathcal{L}_{\text{lat}}^{\text{app}} \\ &\quad + 0.1\,\mathcal{L}_{\text{occ}} + 10^{-6}\,\mathcal{L}_{\text{KL}}, \qquad (8)\\ \mathcal{L}_q &= \mathcal{L}_{\text{rgb}}^{\text{src}} + \tfrac{1}{4}\textstyle\sum_v \mathcal{L}_{\text{rgb}}^{v} \\ &\quad + 0.3\left(\mathcal{L}_d^{\text{src}} + \tfrac{1}{4}\textstyle\sum_v \mathcal{L}_d^{v}\right) + 0.05\,\mathcal{L}_n, \qquad (9)\end{aligned}$$

where v runs over the four supervising target cameras. We use

$$\mathcal{L}_{\text{rgb}} = \|\hat{I} - I\|_1 + 0.2(1 - \mathrm{SSIM}) + \lambda_{\text{LPIPS}}\,\mathrm{LPIPS}_{\text{VGG}}$$

on images composited over the mean-colour background after stored-texture replacement.

$\mathcal{L}_d$ is an L_1 loss on log depth against the per-camera depth estimate, restricted to pixels inside the scene cube and covered by surfels. $\mathcal{L}_n$ is the 2DGS normal-consistency term [13]. $\mathcal{L}_{\Delta t} = \mathrm{mean}_{a<b}\|(\hat{I}_a - \hat{I}_b) - (I_a - I_b)\|_1$ penalizes temporal differences that do not match the ground truth. $\mathcal{L}_{\text{lat}}$ and $\mathcal{L}_{\text{lat}}^{\text{app}}$ are the relative squared errors between slot-decoded and per-frame latents and appearance codes. For the appearance code, the loss uses a random subset of

Table 9. **Number of temporal slots** K on MultiCamVideo (404 test clips, full image). All variants start from the same checkpoint and are trained for 12k steps without appearance codes; only K differs. Dynamic-slot size is the mean number of 32-bit values per clip. ΔPSNR is the per-clip paired difference to K=2 (mean ± 95% CI).

K	Dyn. slot #Floats	PSNR↑	SSIM↑	LPIPS↓	ΔPSNR
2	**21.1K**	16.76	*0.492*	*0.475*	–
3	*31.7K*	*16.78*	**0.493**	*0.475*	$+0.02 \pm 0.01$
10	105.6K	**16.80**	**0.493**	**0.474**	$+0.04 \pm 0.01$

65,536 rows. $\mathcal{L}_{\text{occ}}$ and $\mathcal{L}_{\text{KL}}$ are the occupancy and KL terms of TRELLIS.2 [17, 48].

Final model. The final model combines object-level motion masks, 4-channel appearance codes with slot-compressed dynamic codes, and training with stored texture.

Optimization. We use AdamW [28] ($\beta = (0.9, 0.95)$, no weight decay), a learning rate of 3×10^{-5} with 200 warm-up steps and cosine decay to 10^{-5}, a $10\times$ learning rate for the slot modules, $\lambda_{\text{LPIPS}} = 0.6$, and global gradient-norm clipping at 1.0. The model has 836M trainable parameters, almost all in the TRELLIS.2 backbone.

B.1. Ablation configurations

Table 10. **Learned-architecture ablation** (404 random-window clips). Each row is trained for 10k steps from a shared 20k-step geometry-only checkpoint (30k steps in total). ΔPSNR gives paired per-clip differences to the corresponding parent configuration (mean ± 95% CI). All rows omit stored texture.

Configuration	#Floats	PSNR↑	SSIM↑	LPIPS↓	ΔPSNR
Geometry only	76.2K	16.76	0.492	0.475	–
+ appearance code	0.87M	16.94	0.500	0.448	$+0.18 \pm 0.04$
+ larger code	3.52M	**16.99**	**0.502**	*0.442*	$+0.06 \pm 0.01$
+ motion mask	3.58M	*16.96*	**0.502**	0.444	-0.04 ± 0.01
Combined config.	0.42M	16.95	*0.501*	**0.430**	$+0.01 \pm 0.03$

The appearance-code configuration adds a 16-channel 128^3 code and sub-voxel colour input. The larger-code configuration increases this to 32 channels and adds a 16-channel 256^3 code. The motion-mask configuration additionally uses object-level dynamic masks. The combined configuration keeps the 16-channel 128^3 code. It adds object-level masks, slot-compressed dynamic appearance, random windows across the 81-frame clips, and $\lambda_{\text{LPIPS}} = 0.6$. The final model further reduces the appearance code to four channels and is trained with stored texture.

Table 11. **State size of Sparc4D** in thousands of 32-bit-equivalent values per 32-frame clip (random-window protocol; first-32 values in parentheses). Occupancy uses empirical entropy divided by 32. The first full frame adds $256^2 \times 3$ values; dynamic texture cost depends on the retained pixel area.

Component	Mean	Median
Static + coarse latent	14.7	12.1
Dynamic temporal slots	22.7	16.3
Static appearance code	27.1	22.8
Dynamic appearance slots	56.7	38.5
Occupancy, static + coarse	6.3	5.1
Dynamic occupancy (32f)	27.8	19.9
Environment map	10.4	0.0
State without stored texture	165.7 (158.8)	129.7 (130.7)
Subtotal with first-frame texture	362.3 (355.4)	326.3 (327.3)
Dyn. source pixels, every 4th fr.	308.8 (302.0)	256.0 (238.1)
Sparc4D (complete state)	671.2 (657.4)	588.0 (568.9)

C. Representation-size accounting

Occupancy is represented as a 32^3 bitmap followed by 8-bit child masks at four finer octree levels. Its cost is the zeroth-order empirical entropy of these symbols. Each environment texel counts as four values. Dynamic-texture masks are not stored, because the decoder recomputes them from the dynamic geometry. Scene-layout metadata such as cube origin and scale are omitted.

Temporal-compression ablation. For Table 3, the per-frame variant bypasses the slots at inference and stores dynamic latent and appearance features at every timestamp, at a cost of

$$N_{\text{pf}} = 32 \sum_{t=1}^{T} |\mathcal{U}_t| + 4 \sum_{t=1}^{T} |\mathcal{U}_t^{128}|, \tag{10}$$

where $\mathcal{U}_t$ and $\mathcal{U}_t^{128}$ are the occupied dynamic cells at time t on the 32^3 latent and 128^3 appearance grids, respectively. With temporal slots, the corresponding cost is

$$N_{\text{slot}} = 64|\mathcal{U}| + 8|\mathcal{U}^{128}|, \tag{11}$$

where $\mathcal{U}$ and $\mathcal{U}^{128}$ are the temporal unions of occupied dynamic cells.

For the combined time-varying feature state, the 10th–90th percentile range of the per-clip compression ratio is $3.3\times$–$7.4\times$.

For MoVieS, the materialized output contains 1,605,632 Gaussians at 32×224^2 input pixels. The reduced-output study in Table 2 counts its native prediction tensors: 14 static parameters and 4×32 query-time motion parameters per Gaussian, or 142 values. For the cross-method tables,

Table 12. **Temporal feature storage** over the 404 random-window test clips. Sizes are mean 32-bit-equivalent values per clip. Compression factors are computed per clip and reported as their median and mean. Total state includes stored texture.

Component	Per-frame	Slots	Median	Mean
Dynamic latent	182.1K	22.7K	8.8×	8.7×
Dynamic appearance	172.5K	56.7K	3.5×	3.8×
Time-varying features	354.5K	79.4K	5.0×	5.2×
Total state	0.96M	0.67M	1.4×	1.4×

we follow the VideoTok4D reporting convention [5]. It materializes the time-conditioned representation at every evaluated timestamp, which gives 719M FP32 values for a 32-frame sequence. For 4DGT we count the predicted Gaussian parameters. For C4G we count the outputs of 2,048 Gaussians at each of the 32 queried timestamps.

Two size conventions. Table 13 adds the stored size, which counts what must be kept to render any observed timestamp. Sparc4D is smallest under this convention. Its dense decoded output, about 0.39M surfels per timestamp, exceeds that of C4G when materialized.

Table 13. **Representation size under two conventions**, in values per 32-frame clip. Materialized sizes use a common 14 values per primitive (position, scale, rotation, opacity and one RGB colour) at each of the 32 timestamps. With its degree-1 spherical-harmonics colour, Sparc4D would count 22 values per surfel (277M). For Sparc4D, materialized sizes are averaged over the 24 clips of Table 14. †Upper bound without lifetime culling. ‡Estimated from the architecture: 12 context frames × 256 tokens × 2,048 channels.

Method	Stored	Materialized
MoVieS	228M	719M
4DGT	26.9M	301M†
C4G	6.3M‡	**0.92M**
Sparc4D	**0.67M**	176M

D. Runtime and memory

Table 14 reports runtime on one A800 GPU with batch size 1, as the median over 24 test clips. Each clip has 32 input frames and is rendered at 32 timestamps from 10 cameras, giving 320 images at 256^2. MoVieS and 4DGT decode inside their renderers, and C4G decodes all timestamps during encoding.

Table 14. **Runtime and memory per clip.** For Sparc4D, rendering includes stored-texture reprojection.

Method	Encode	Decode	Render	Per image	Memory	Params
MoVieS (bf16)	2.13 s	–	2.42 s	7.6 ms	15.7 GB	1041M
4DGT	1.10 s	–	7.55 s	23.6 ms	9.8 GB	1223M
C4G	2.29 s	–	0.09 s	0.3 ms	5.3 GB	1014M
Sparc4D (fp32)	0.40 s	1.70 s	1.84 s	5.8 ms	4.8 GB	836M

E. Evaluation details

E.1. Split and camera separation

We compute a DINOv2 environment feature for each scene and cluster the 3,400 scenes with k-means ($k = 120$). Test scenes are drawn from clusters assigned to the test side, and each has cosine similarity below 0.62 to every scene in the remaining clusters. This gives 101 test scenes, each rendered at four focal lengths (404 clips).

The remaining 3,299 scenes (13,196 clips) form the training set.

The four focal lengths are 18, 24, 35 and 50 mm. The ten cameras coincide at frame 0 and separate over time. Measured over 404 test clips, mean pairwise separation is 0.93 m at frame 31 and 2.10 m at frame 80. In first-32, 70% of target images are within 0.5 m of the source camera; in random windows, 35% are. The two protocols therefore cover different view-separation regimes.

E.2. Visibility masks and image metrics

MultiCamVideo provides no co-visibility masks. We construct a shared mask from estimated depth and foreground segmentation, independently of each reconstruction method. A target pixel is back-projected using its estimated depth. Depth scale is aligned to the source using a median depth ratio before projecting the point into source frames.

A background pixel is accepted if at least one source frame sees a background surface at the projected position with depth agreement within 10%. An RMBG foreground pixel is accepted only through the source frame at the same time. This restriction avoids treating a moving point as static across frames. Sky and points beyond 40 m are checked by direction. The resulting masks cover on average 83% of target pixels for first-32 and 64% for random windows.

For mask M and RGB intensities in $[0, 1]$, we use

$$\mathrm{mPSNR} = -10 \log_{10} \frac{\sum_{\mathbf{x}} M(\mathbf{x}) \|\hat{I}(\mathbf{x}) - I(\mathbf{x})\|^2}{3 \sum_{\mathbf{x}} M(\mathbf{x})}. \quad (12)$$

mSSIM and mLPIPS average their spatial metric maps inside the mask. Images with less than 1% observed pixels are omitted from masked averages, about 2% of random-window images. Masked and uniform-fill scores use stored 8-bit renderings.

For region analysis, current-frame dynamic input voxels are rendered as camera-facing disks into each target camera. We threshold opacity at 0.5 and dilate by two pixels. The static region lies inside the scene cube. Dynamic-region LPIPS uses the region's bounding box, padded by eight pixels and resized to 128^2.

E.3. Baseline configurations

MoVieS receives all 32 frames at 224^2, together with ground-truth cameras and timestamps. 4DGT receives all 32 frames at 560^2 and ground-truth cameras. Its timestamps use frame index divided by five rather than the dataset's physical rate of 15 fps. We select this setting by source-view reconstruction quality.

C4G is pose-free and receives 12 frames spread uniformly over the window at 224^2, matching its training context count. We align its arbitrary reconstruction scale to the estimated depth of its first context frame using a median ratio. Target cameras are expressed relative to that frame. The fine-tuned model also keeps C4G's architecture and 12-frame input.

E.4. DyCheck preprocessing

We use the *apple*, *block*, *paper-windmill*, *space-out*, *spin*, *teddy* and *wheel* sequences. The handheld video supplies 32 input frames from every second timestamp of a non-overlapping 64-timestamp window. At 30 fps this gives the 15 fps input sampling used in training. Targets are fixed-camera images present in the official validation split at those input times. Windows with no target image are dropped.

We centre-crop the half-resolution 360×480 images to 360^2 and resize to 256^2. Images are resampled to the intrinsics of the first handheld frame in each window. Official co-visibility masks use nearest-neighbour resampling. Source depth and foreground masks are prepared as for MultiCamVideo. The baselines use Appendix E.3's settings; all receive known cameras except pose-free C4G.

Table 15. **mPSNR per DyCheck sequence.** The texture-free variant is an ablation of Sparc4D.

Sequence	MoVieS	w/o stored texture	Sparc4D
apple	*15.54*	**15.62**	15.38
block	14.84	**15.37**	*15.06*
paper-windmill	**15.79**	*14.73*	14.44
space-out	*14.16*	**14.36**	14.07
spin	**13.49**	*13.48*	13.11
teddy	11.38	**12.76**	*12.23*
wheel	**12.94**	*11.72*	11.46

E.5. Neu3D preprocessing

We evaluate six Neu3D [20] scenes using 24 non-overlapping 32-frame windows. For each window, a fixed central camera supplies the monocular input. At the same timestamps, the targets are cam00 and eight cameras sampled at approximately uniform index intervals, for nine target cameras in total. We report full-image PSNR, SSIM and LPIPS over all nine targets, and PSNR separately for cam00. Cam00 is the conventional Neu3D test camera and is approximately 10 cm from the input camera. All methods use the settings in Appendix E.3.

E.6. Qualitative selection

For each MultiCamVideo protocol, clips are ranked by how close their paired PSNR difference between Sparc4D and MoVieS lies to the median difference. We use the target view at timestamp 16 from the camera at median distance from the source. The figures show the two highest-ranked clips from distinct scenes in which this view has at least 60% observed pixels and at least 1,024 observed foreground pixels. The dynamic crop is centred on the largest observed foreground component. The static crop is the most textured observed background window. Both crops are computed from the ground truth and the visibility mask, and all methods share them.

F. Selected zero-shot reconstructions

Figures 5 and 6 compare real-video novel views using the same model configurations as Table 5. These examples are selected to show visible strengths of Sparc4D in subject reconstruction and local detail; the aggregate results in that table cover all evaluated windows. Each case uses the same target camera, timestamp and crop for all methods. The figures show the cached RGB renderings without visibility masking or image enhancement. The scene, source/target cameras and original frame index identify each case; t is the timestamp within its 32-frame input clip.

G. Depth consistency

MultiCamVideo does not provide ground-truth depth. We compare rendered target-view depth with per-camera DA3 estimates using AbsRel, RMSE and δ_1, the fraction of pixels whose depth ratio is within 1.25 [9]. We also report median-scaled AbsRel and δ_1. Each method is evaluated where it renders a surface and the reference is valid; coverage is reported alongside the errors.

Rendered depth from Sparc4D is more consistent with the DA3 reference than the evaluated baselines. Because DA3 also supplies our input depth and depth supervision, this comparison measures agreement with DA3.

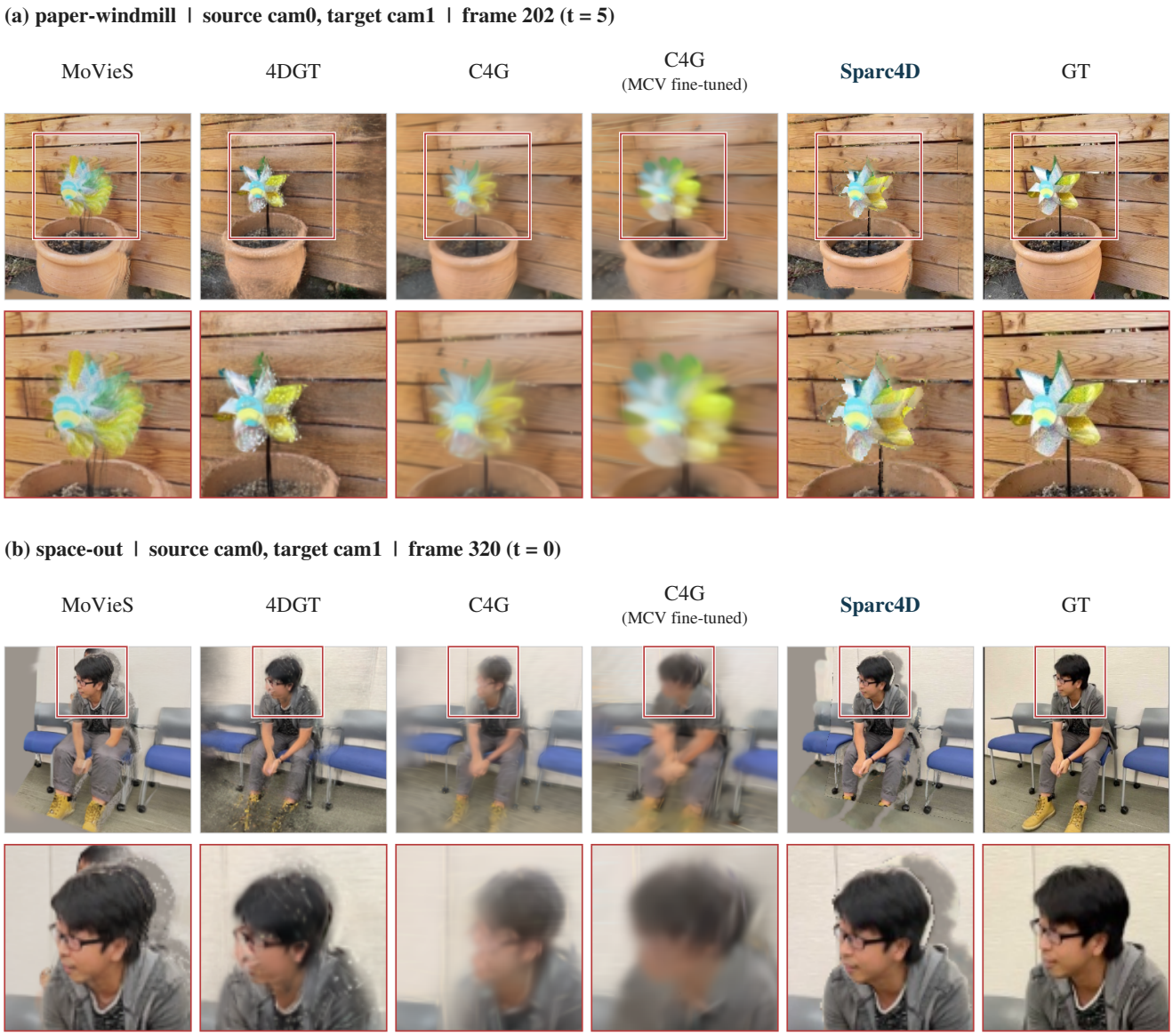


Figure 5. **Selected zero-shot novel-view reconstructions on DyCheck.** The paper-windmill and space-out examples show pinwheel structure and facial/clothing detail, respectively. For each example, full images are followed by enlarged views of the same red-box region. C4G (MCV fine-tuned) uses the synthetic-data adaptation reported in Table 5; Sparc4D is the complete model with stored texture. All methods are evaluated without real-data fine-tuning.

H. Additional reconstruction diagnostics

I. Downstream manipulation protocol

Setup. The autoencoder receives 4,000 adaptation steps on ManiSkill scenes before policy training. Four 256^2 RGB-D views, sampled on a ring around the workspace, are fused into a fixed 2.4 m cube. Static features pool the first three frames of each observation window. Simulator depth and robot-link silhouettes are used; SAM 2 tracks objects causally from boxes obtained by clustering non-robot points above the table, without object-pose or object-segmentation labels. The policy also observes the end-effector pose and gripper state. It is trained for 60,000 steps with batch size 64. It predicts eight-step action chunks with ten Euler steps and replans after each chunk. The additional 2,000 demonstrations randomize motion-planning waypoints and initial joint-motion prefixes.

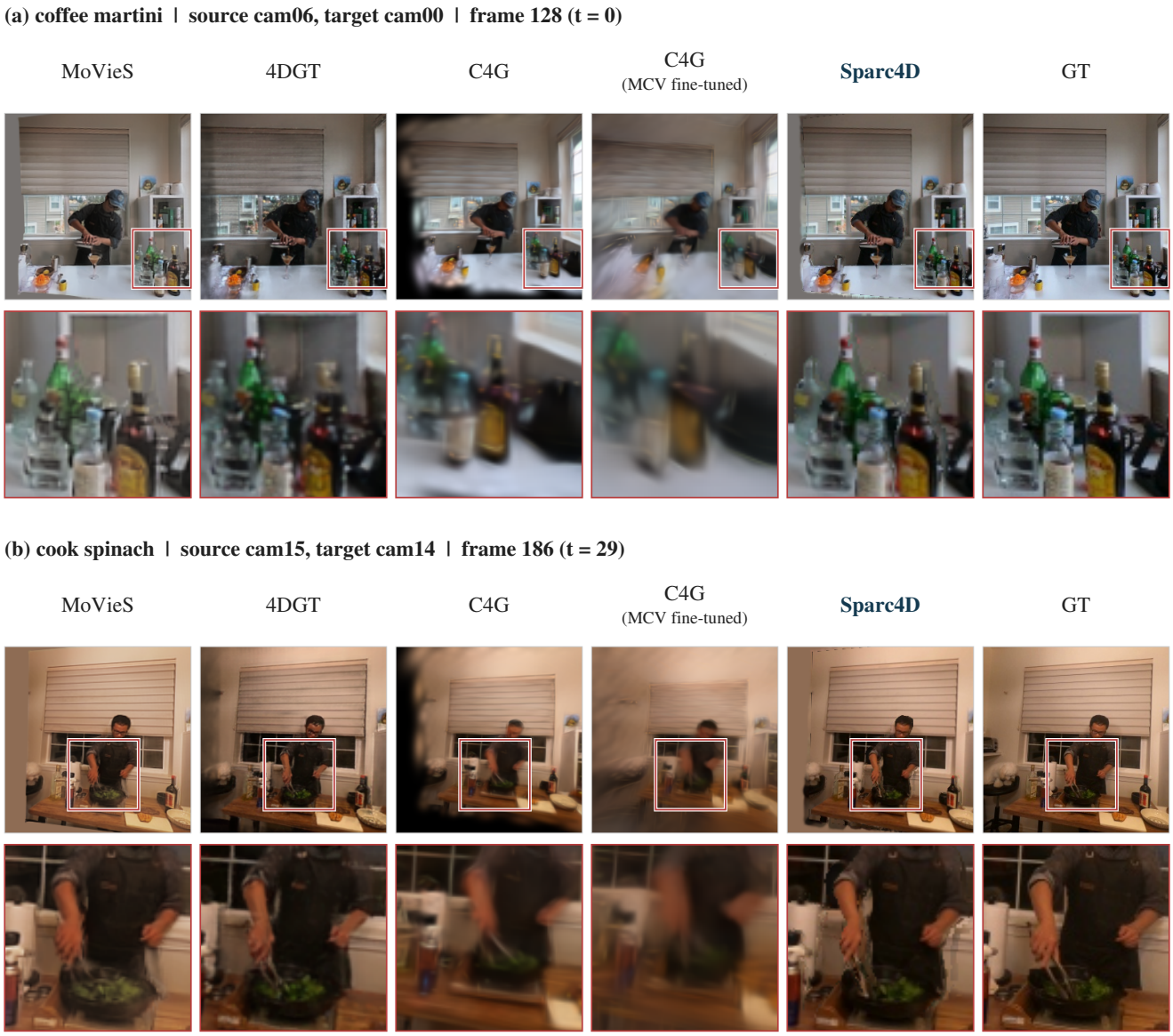


Figure 6. **Selected zero-shot novel-view reconstructions on Neu3D.** The coffee-martini example uses the conventional cam00 target, while cook-spinach uses cam14. The crops show bottle/countertop detail and the hands/cooking region. Full images and enlarged crops share camera, time and spatial coordinates across methods. The model settings are the same as in Fig. 5.

Evaluation. Success is measured within 200 control steps over 100 initial states and three sampling seeds per trained policy. Li et al. [24] report results with fixed 512^2 cameras and joint-position control; ours use randomized cameras and end-effector increments.

Scene size. Our counts average 306 held-out windows: 425.9 static and 235.5 dynamic cells per timestamp, or 661.4 in total. The three-frame policy context contains 1,132.5 cells. The same encoder and reference windows are used for both data budgets. The approximately 5,700 cells for Li et al. come from three initial states with their fixed cameras. These counts include scene-latent cells only.

Table 16. **Rendered depth vs. the per-camera depth estimate** at target views (test set, per-image average). "aligned": per-image median scaling. C4G-FT: C4G fine-tuned on MultiCamVideo.

					aligned		
	Method	AbsRel↓	δ_1 ↑	RMSE↓	AbsRel↓	δ_1 ↑	cov.
First-32	4DGT	0.624	0.141	8.56	0.344	0.621	*0.92*
	C4G	0.505	0.538	6.34	0.476	0.586	0.91
	C4G-FT	*0.267*	*0.592*	*5.16*	0.291	0.598	**0.98**
	MoVieS	0.561	0.513	6.21	*0.250*	*0.717*	*0.92*
	Sparc4D	**0.157**	**0.848**	**2.66**	**0.115**	**0.885**	0.89
Random	4DGT	0.640	0.155	8.40	0.370	0.598	0.81
	C4G	0.543	0.522	6.12	0.450	0.596	*0.84*
	C4G-FT	*0.356*	*0.524*	*5.43*	0.346	0.545	**0.97**
	MoVieS	0.615	0.505	5.84	*0.272*	*0.704*	0.75
	Sparc4D	**0.259**	**0.784**	**2.92**	**0.170**	**0.831**	0.75

Table 17. **Static and dynamic regions**, first-32 protocol. Regions are defined in Appendix E.2.

	Dynamic		Static
Method	PSNR↑	LPIPS↓	PSNR↑
4DGT	16.19	0.345	18.56
C4G	14.32	0.434	15.97
MoVieS	*19.02*	**0.150**	21.79
w/o stored texture	17.53	0.276	21.43
w/o dynamic texture	18.36	0.201	*25.41*
4 full frames, no dyn. texture	18.47	0.193	**25.53**
Sparc4D	**19.32**	*0.153*	*25.41*

Table 18. **Rendering ladder** on random windows (8 timestamps per clip). Only the Sparc4D row uses stored texture. Rows 1–3 directly render input pixels or voxels as mean-colour disks. "dyn." denotes the dynamic region and "stat." the static region inside the scene cube.

	Source, dyn.		Target, dyn.		Target, stat.
Rendering	PSNR↑	LPIPS↓	PSNR↑	LPIPS↓	PSNR↑
Current-frame pixels	27.27	0.048	14.90	0.338	14.75
Input voxels, 512^3	18.12	0.272	14.67	0.419	15.39
Input voxels, dyn. 1024^3	20.15	0.212	14.81	0.387	15.39
Learned state, per-frame dyn.	20.75	0.209	15.88	0.344	18.37
Learned state, temporal slots	20.23	0.218	15.88	0.347	18.37
Sparc4D (with stored texture)	–	–	16.36	0.264	19.21
MoVieS	21.93	0.089	16.76	0.257	18.36

Table 19. **PSNR by frame index within the window** (random windows, mean over 9 target cameras, computed from 8-bit renders).

t	0	4	8	16	24	31
w/o stored texture	17.71	17.44	17.18	16.72	16.38	16.11
w/o dynamic texture	**19.67**	18.18	17.76	17.10	16.66	16.33
Sparc4D	**19.67**	*18.49*	*17.92*	*17.17*	16.68	16.30
All 32 full frames	*19.60*	**18.50**	**17.94**	**17.20**	*16.71*	*16.36*
MoVieS	18.18	17.93	17.67	*17.17*	**16.81**	**16.50**